



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE CIÊNCIAS EXATAS E DA TERRA
DEPARTAMENTO DE INFORMÁTICA E MATEMÁTICA APLICADA
PROGRAMA DE PÓS-GRADUAÇÃO EM SISTEMAS E COMPUTAÇÃO



Reprodução de experimentos de medição de energia em linguagens de programação: C++ e Python

Aluno: Maycon Nazario
Prof. Dr. Sergio Medeiro

Natal-RN
2026

Dois artigos: um ranking e a sua revisão

Pereira et al. (2021)

- 27 linguagens, 10 problemas do Computer Language Benchmarks Game
- Medidor em C que lê os contadores RAPL antes e depois de cada execução
- Resultado: um ranking de eficiência energética
- Convertido para a base C++: Python consome 56,63 vezes a energia de C++
- Afirma que a linguagem mais rápida nem sempre é a mais econômica

van Kempen et al. (2024)

- Mesmas implementações, medidor corrigido, contadores de desempenho
- Aponta no trabalho anterior erros do medidor e paralelismo desigual entre implementações
- Modelo causal: a potência depende do número de núcleos ativos e da atividade de memória
- Conclui que a linguagem só afeta a energia por meio do tempo de execução

Objetivo, escopo e regra de leitura

- Reproduzir os experimentos dos dois artigos, restritos a C++ e Python, na mesma máquina
- Comparar cada reprodução com os dados publicados pelos autores e com os valores impressos nos artigos
- Cruzar as duas reproduções pela razão Python/C++ por benchmark
- Avaliar as críticas de van Kempen et al. a Pereira et al. com dados próprios

Três rótulos nas tabelas

artigo: valor impresso pelos autores

restrito: dados brutos publicados pelos autores, medidos na máquina deles, filtrados para C++ e Python

reprodução: dados medidos nesta atividade. A comparação justa é restrito contra reprodução.

Ambiente comum às duas reproduções

- Intel Core i7-9700, oito núcleos; Ubuntu 24.04.4, Linux 6.8
- Contêineres Docker construídos dos repositórios dos autores, com Clang 19 e CPython 3.12.6
- Artefatos dos autores com adaptações mínimas: compilador e interpretador nos Makefiles, modelo do processador no medidor RAPL do Pereira; imagem fixada em Ubuntu 24.04 no van Kempen
- Análise validada primeiro sobre os dados publicados, depois aplicada à reprodução
- Código, dados e scripts: github.com/mayconfreitas/reproduction-energy-languages

3,0 GHz, sem turbo, oito núcleos

Campanha de Pereira et al.

4,7 GHz, com turbo, oito núcleos

Configuração padrão de van Kempen et al.

800 MHz, um núcleo

Frequência mínima de van Kempen et al.

Quantos núcleos cada implementação ocupa

Benchmark	mec. C++	núcleos C++	mec. Python	núcleos Python
binary-trees	OpenMP	5,94	multiprocessing	6,52
fannkuch-redux	OpenMP	7,96	multiprocessing	7,88
fasta	threads	–	nenhum	1,00
k-nucleotide	threads	5,43	multiprocessing	7,65
mandelbrot	OpenMP	7,94	multiprocessing	7,98
n-body	só flag	1,00	nenhum	1,00
regex-redux	OpenMP	1,69	multiprocessing	2,26
reverse-complement	threads	1,67	multiprocessing	1,49
spectral-norm	OpenMP	7,89	multiprocessing	3,99

- Nenhum dos dois artigos publica esse número por benchmark
- O código indica a intenção; o contador mede o resultado. Discordam em regex-redux C++ (1,7), reverse-complement Python (1,5) e n-body C++ (1,0)
- A divisão entre paralelos e sequenciais é por benchmark e igual nas duas linguagens, salvo spectral-norm: 7,9 núcleos em C++ e 4,0 em Python

Tabela 1: mecanismo no código e número médio de núcleos ativos medido pelo task-clock (mediana de 21 execuções, configuração padrão, reprodução).

Reprodução de Pereira et al. (2021)

Medidor original, dez execuções, energia CORE + DRAM, média aparada

P Validação e dois problemas nos dados

- A análise foi aplicada primeiro aos CSV publicados pelos autores
- A energia do artigo é CORE + DRAM: é a soma que reproduz sua Tabela 3 (41,25 J contra 41,23 J em binary-trees C++); PACKAGE + DRAM daria de 15% a 25% a mais
- CORE é 55% a 89% do PACKAGE no i5-4460 e 97% a 99% no i7-9700

pidigits em Python produz saída errada

O programa espera divisão inteira em div do gmpy2; desde a versão 2.1, div é divisão real. Roda sem erro, gera dígitos incorretos e leva 360 s em vez de 1 s. Retirado das duas reproduções.

P Energia e tempo por benchmark

Tabela 2: dados restritos (i5-4460, CPython 3.6)

Benchmark	C++ E (J)	C++ T (ms)	C++ E/T	Py E (J)	Py T (ms)	Py E/T
binary-trees	41,25	1 108	0,037	1 794	44 837	0,040
fannkuch-redux	219,91	6 116	0,036	12 789	279 542	0,046
fasta	34,94	1 153	0,030	1 059	74 059	0,014
k-nucleotide	111,10	3 814	0,029	1 601	38 925	0,041
mandelbrot	32,44	857	0,038	7 786	162 831	0,048
n-body	57,29	3 770	0,015	7 874	558 883	0,014
regex-redux	175,42	10 610	0,017	162	7 127	0,023
reverse-complement	6,16	223	0,028	32	1 493	0,022
spectral-norm	17,23	665	0,026	6 422	137 717	0,047

Tabela 3: reprodução (i7-9700 , CPython 3.12)

Benchmark	C++ E (J)	C++ T (ms)	C++ E/T	Py E (J)	Py T (ms)	Py E/T
binary-trees	28,42	1 150	0,025	591	18 448	0,032
fannkuch-redux	167,28	4 884	0,034	4 683	121 484	0,039
fasta	19,97	1 051	0,019	316	37 732	0,008
k-nucleotide	47,62	1 927	0,025	789	21 337	0,037
mandelbrot	16,51	475	0,035	2 973	71 634	0,041
n-body	19,69	2 753	0,007	2 237	282 762	0,008
regex-redux	103,78	10 617	0,010	89	7 254	0,012
reverse-complement	3,93	316	0,012	20	1 779	0,011
spectral-norm	5,75	218	0,026	1 855	80 126	0,023

- Energia menor na reprodução em todos os 18 pares
- C++: tempo cai para um terço ou metade nos paralelos (spectral-norm, mandelbrot, k-nucleotide) e fica igual ou maior em binary-trees, regex-redux e reverse-complement
- Python: tempo cai para 41% a 58% nos benchmarks mas não cai em regex-redux e reverse-complement, que rodam em bibliotecas em C
- Dois fatores, nenhum deles a linguagem: interpretador (3.6 para 3.12) e paralelismo das implementações,

P Energia e tempo por benchmark: C++

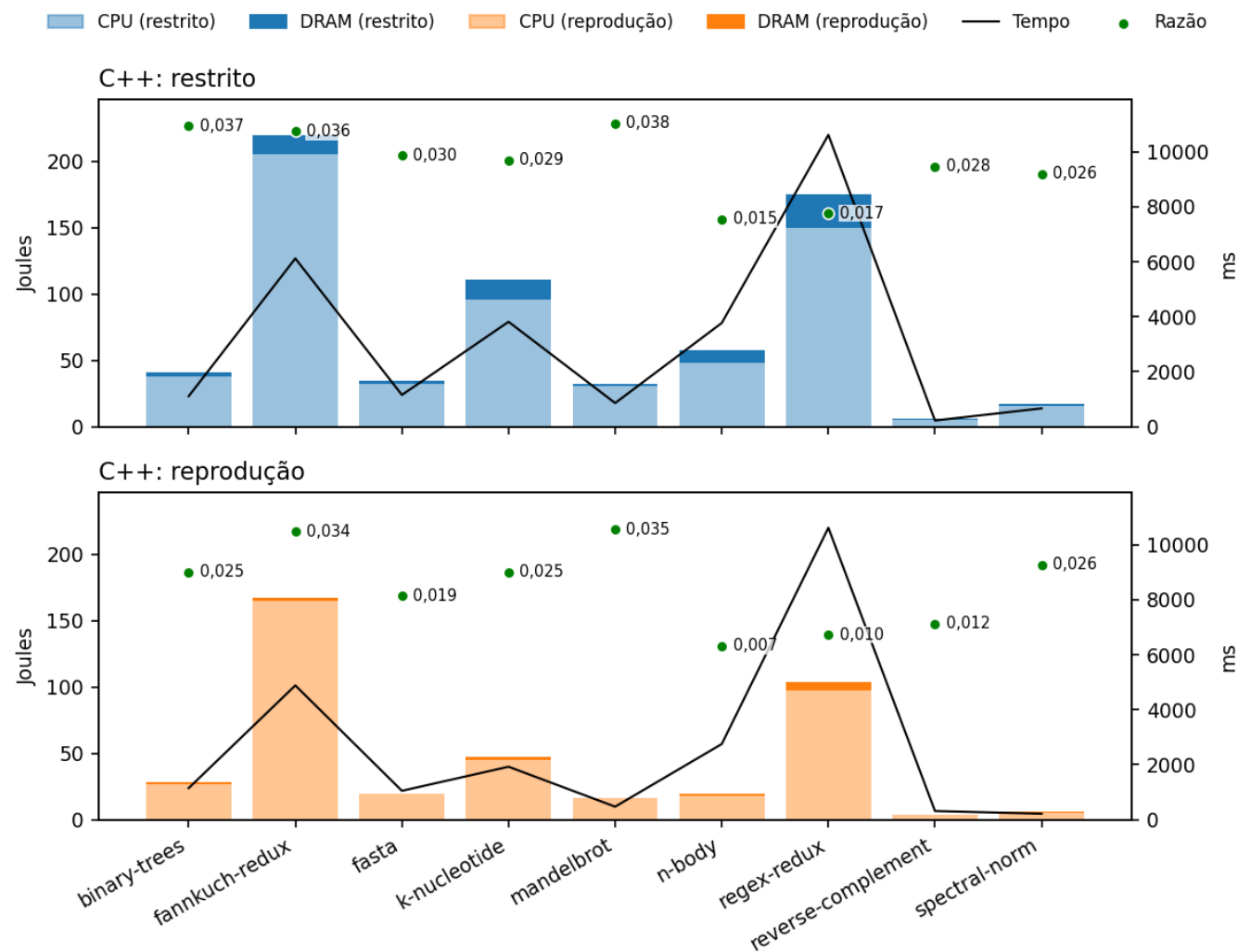


Figura 1: energia de CPU e DRAM (barras), tempo (linha) e razão energia/tempo (pontos); restrito acima, reprodução abaixo, mesma escala.

Reprodução de experimentos de medição de energia: C++ e Python

- Mesma escala nos dois painéis: as barras encolhem mais que a linha do tempo
- Em binary-trees, regex-redux e reverse-complement, a energia cai 30% a 40% com o mesmo tempo: menos potência no mesmo intervalo
- Os pontos da razão ficam mais altos nos benchmarks paralelos

P Energia e tempo por benchmark: Python

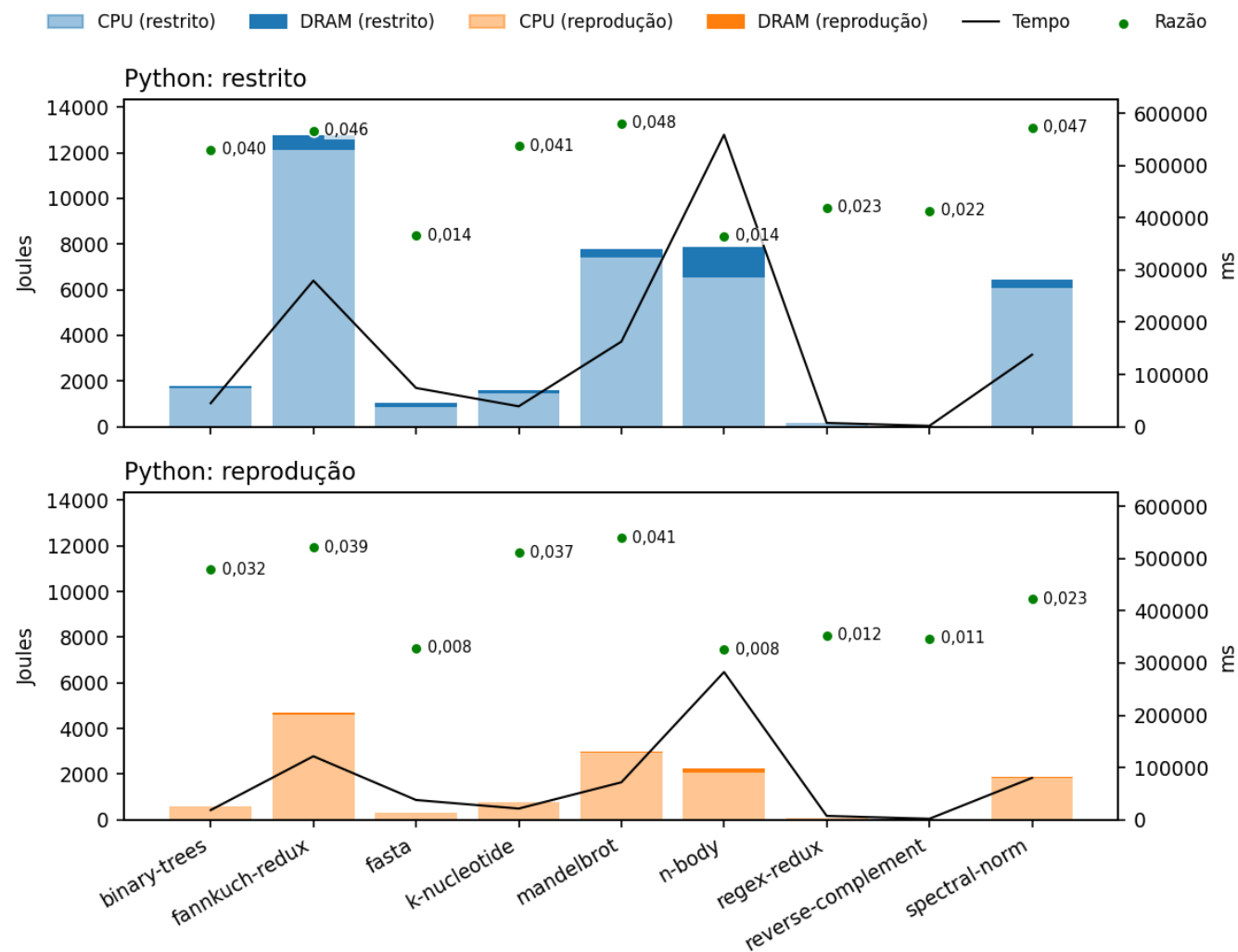


Figura 2: mesmo formato da Figura 1, para Python.

- O tempo de Python cai cerca de metade nos benchmarks de cálculo
- regex-redux e reverse-complement quase não mudam: o trabalho é feito em C dentro do runtime
- O ganho vem do interpretador, não da máquina

P Energia de DRAM: a ordem se reproduz, a proporção não

Benchmark	restr. C++	restr. Python	repr. C++	repr. Python
binary-trees	3,7	127	1,3	25,8
fannkuch-redux	14,8	675	2,8	69,0
fasta	2,9	179	0,7	21,5
k-nucleotide	15,2	121	2,4	33,4
mandelbrot	2,1	393	0,3	40,8
n-body	9,1	1350	1,6	161
regex-redux	26,1	18,0	6,5	5,0
reverse-complement	0,9	4,3	0,4	1,6
spectral-norm	1,6	333	0,1	45,7
média	8,5	356	1,8	44,8

Tabela 4: energia de DRAM por benchmark (J).

88,9 %

fração da CPU no artigo (C++)

95,1 %

fração da CPU na reprodução (C++)

- Fração da CPU = $\text{CPU} / (\text{CPU} + \text{DRAM})$, média dos nove benchmarks (Tabela 5)
- As duas energias caíram, mas a DRAM caiu mais (4,8 e 8,0 vezes contra 1,6 e 2,8), e a fatia da CPU subiu

P Agregados: a ordem se mantém, o fator não

Origem	Linguagem	Energia (J)	Tempo (ms)	Energia norm.	Tempo norm.
reprodução	C++	45,88	2 599	1,00	1,00
reprodução	Python	1 505,95	71 395	32,82	27,47
artigo	C++	77,50	3 155	1,00	1,00
artigo	Python	4 390,00	145 178	56,63	46,09

Tabela 6: médias dos nove benchmarks, normalizadas por C++ (valores do artigo convertidos da base C).

Origem	C++	Python
reprodução (9 benchmarks)	0,87	0,87
restrito (9 benchmarks)	0,97	0,93
artigo (10 benchmarks)	0,97	0,93

Tabela 7: correlação de Spearman entre energia e tempo; "muito forte" em todos os casos (classificação de Rea e Parker adotada pelo artigo).

56,6×

energia Python/C++ no artigo

32,8×

na reprodução

- Com as mesmas implementações, o fator cai para pouco mais da metade
- Tirar pidigits não muda o Spearman do artigo; a queda para 0,87 é da medição
- Nos nove benchmarks e nos dois conjuntos, a linguagem mais rápida é também a mais econômica

Reprodução de van Kempen et al. (2024)

21 execuções e mediana, PKG + DRAM, duas configurações

K Protocolo, ajustes de escopo

- Duas configurações: padrão, com todos os núcleos, e frequência mínima com turbo desligado em um núcleo
- Única alteração nos arquivos: Dockerfile reduzido a C++ e Python. Imagem fixada em Ubuntu 24.04, ferramenta RAPL compilada na imagem.
- fasta não tem versão C++ no repositório dos autores; pidigits também está quebrado nos dados publicados (445 vezes mais lento em Python)
- PPO, o domínio dos núcleos (o CORE da outra reprodução), é 98% a 100% de PKG na configuração padrão

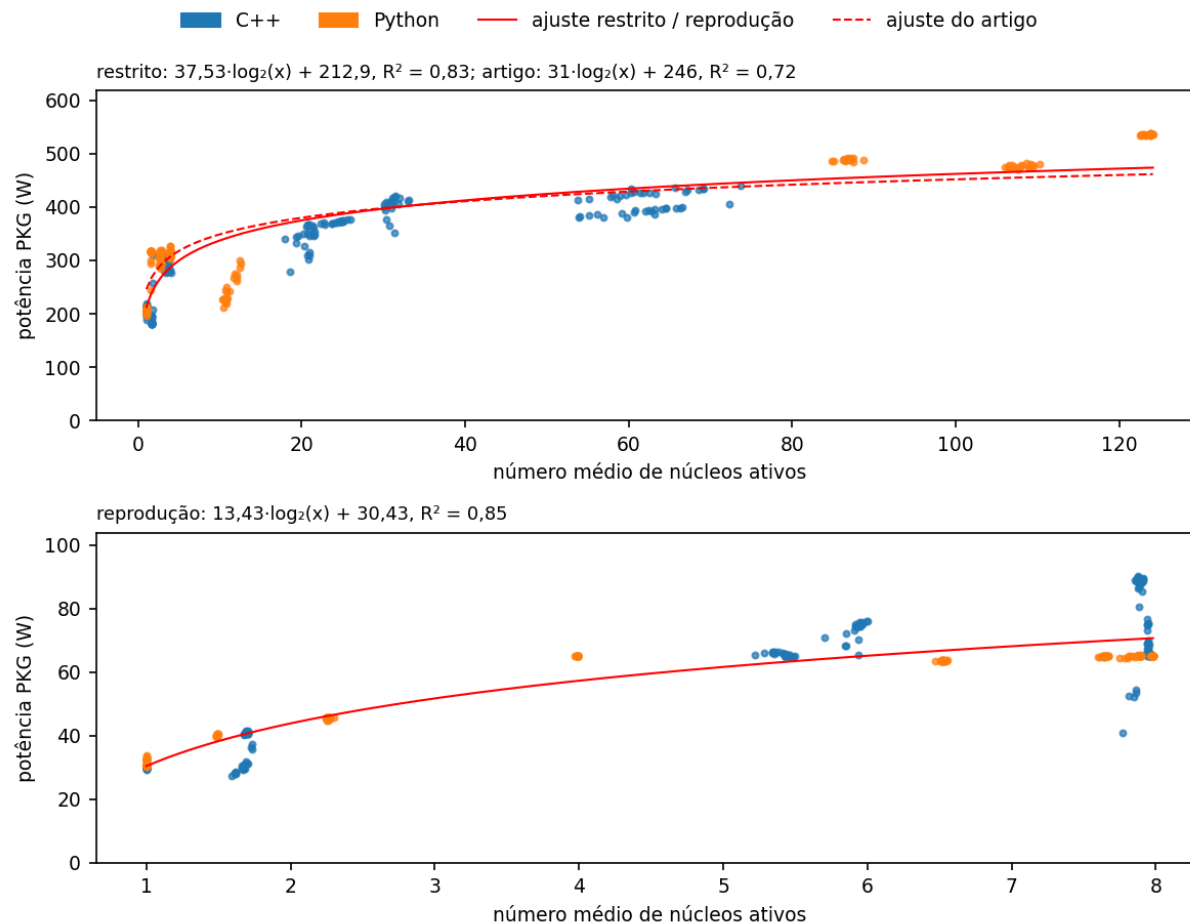
K A frequência fixa, todos os pares têm a mesma potência

Origem	Média (W)	Desvio (W)	CV (%)	Mínimo (W)	Máximo (W)	Execuções
artigo	189,80	0,50	0,26	–	–	–
restrito	189,69	0,38	0,20	189,12	191,00	51
reprodução	2,23	0,11	5,00	1,95	2,42	51

Tabela 8: potência PKG + DRAM na frequência mínima em um núcleo, sobre todas as execuções.

- O recorte a C++ e Python não altera o resultado dos autores: 189,69 contra 189,8 W
- Na reprodução, todas as execuções ficam em $2,23 \pm 0,11$ W: nenhum par se destaca
- A dispersão é maior (5% contra 0,26%) porque o servidor mede o pacote de 128 núcleos, e a variação entre programas se dilui na potência de fundo
- A afirmação do artigo se sustenta a menos de 5%: energia proporcional ao tempo

K Potência em função dos núcleos ativos



Origem	a (W)	b (W)	R ²	P(2)/P(1)
artigo	31,00	246,00	0,72	1,13
restrito	37,53	212,85	0,83	1,18
reprodução	13,43	30,43	0,85	1,44

Tabela 9: ajuste $a \cdot \log_2(x) + b$. $P(2)/P(1) = (a + b)/b$ é o ganho mínimo de vazão para paralelizar compensar.

Figura 3: potência PKG contra núcleos ativos, formato da Figura 8a do artigo.

K Potência de DRAM em função das faltas de LLC

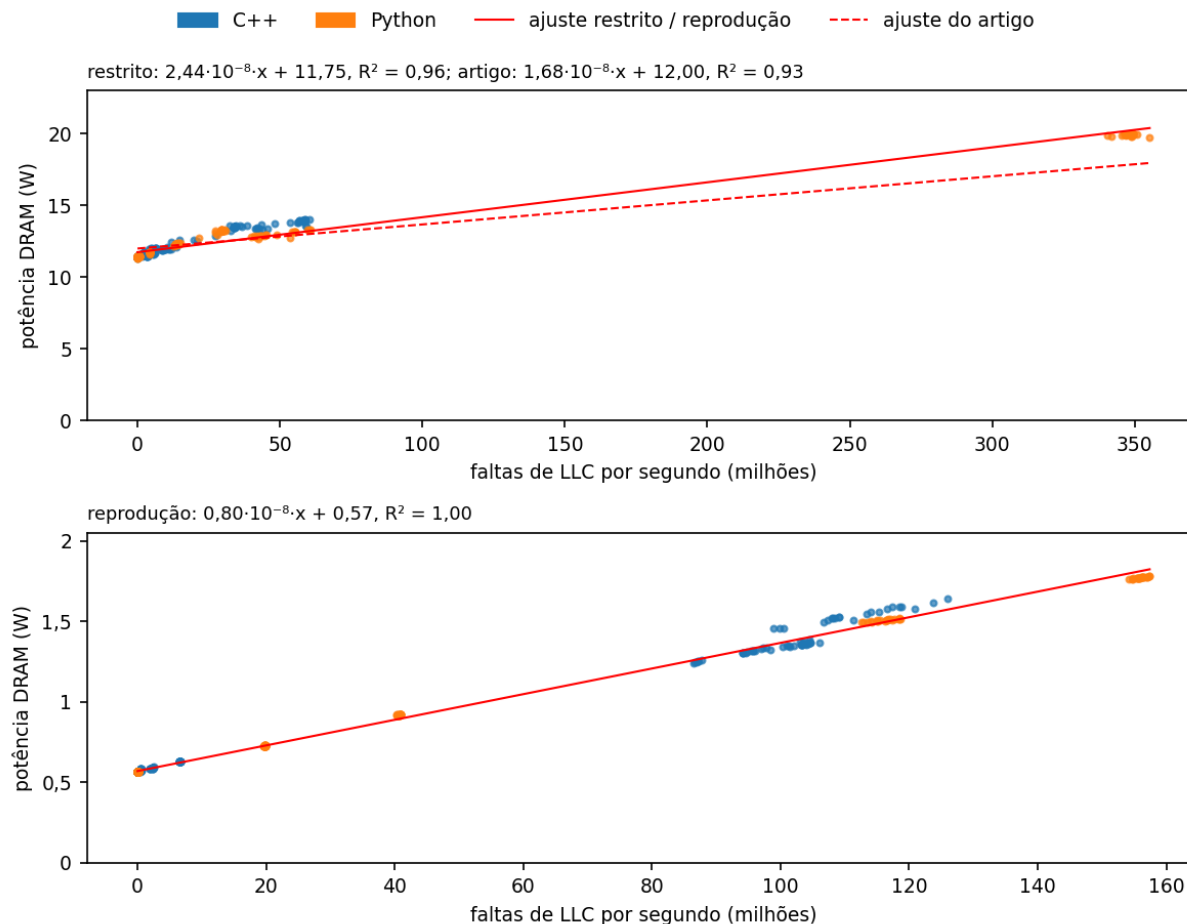
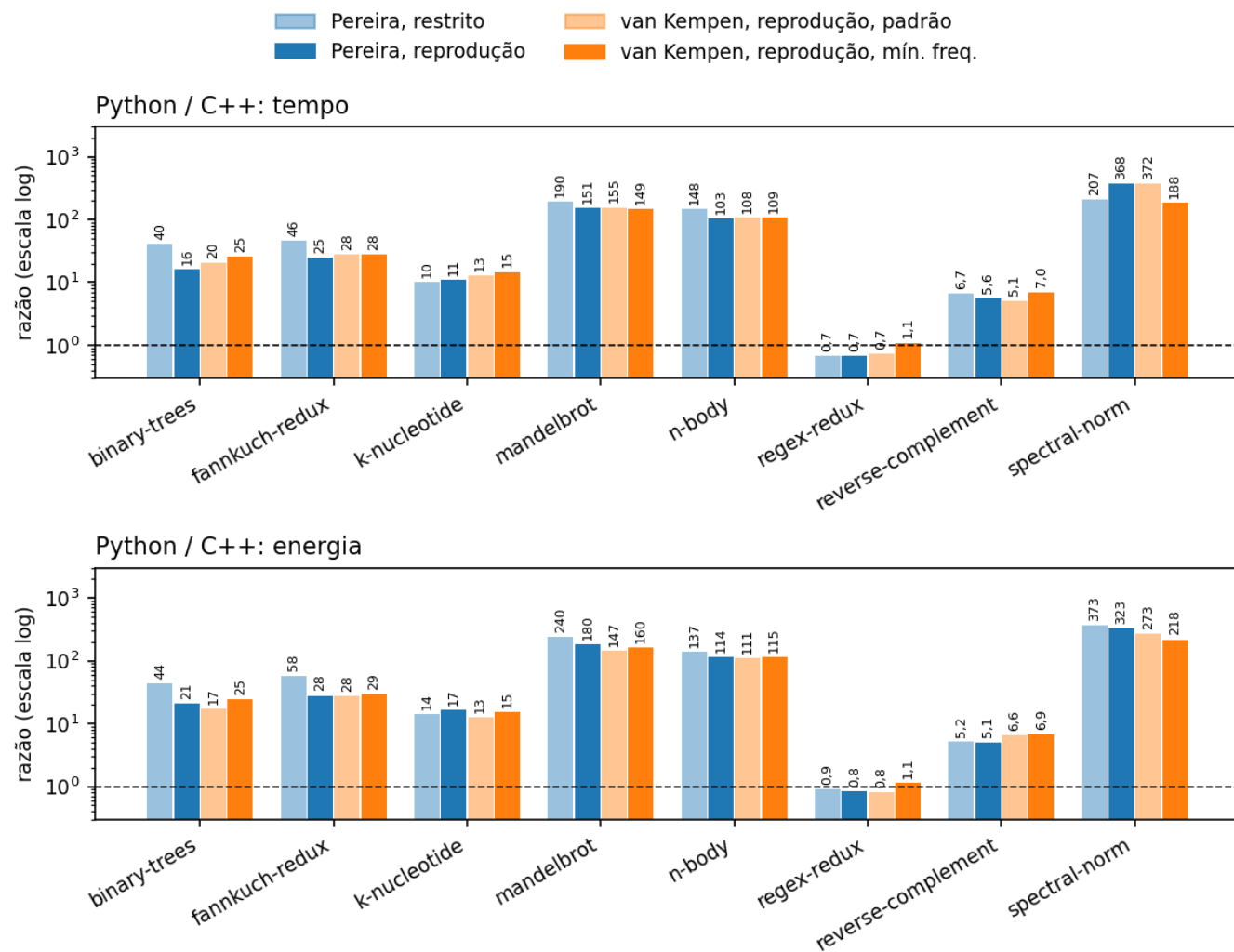


Figura 4: potência de DRAM contra faltas de LLC por segundo, formato da Figura 8b do artigo.

Config.	Origem	a	b (W)	R ²	DRAM/PKG (%)
padrão	artigo	1,68	12,00	0,93	2,0 a 8,0
padrão	restrito	2,44	11,75	0,96	2,2 a 7,5
padrão	reprodução	0,80	0,57	1,00	0,6 a 4,5
mín. freq.	restrito	—	—	—	6,3 a 7,1
mín. freq.	reprodução	—	—	—	30,8 a 44,9

Tabela 10: ajuste $a \cdot x + b$, a em W.

c Razão Python/C++ em quatro conjuntos



- Na mesma máquina, três estados do processador dão barras de mesma altura
- A razão só muda entre estados onde as versões ocupam números diferentes de núcleos
- spectral-norm: 188 em um núcleo, 368 e 372 com todos os núcleos
- regex-redux: Python mais rápido com todos os núcleos (0,68 e 0,73), mais lento em um (1,10)

Figura 5: razões de tempo e de energia em escala logarítmica; os três últimos conjuntos foram medidos na mesma máquina.

C As críticas de van Kempen et al. à luz das reproduções

Crítica a Pereira et al.	Resultado	Evidência
Erros do medidor	Confirmada	Três leituras negativas nos dados restritos e uma na reprodução, por estouro do contador de 32 bits
Paralelismo desigual	Confirmada	Núcleos de 1,0 a 8,0 entre benchmarks (Tabela 1); é o que separa a razão do servidor da razão do desktop
Resposta à RQ2 (a mais rápida nem sempre é a mais econômica)	Confirmada	Nos oito benchmarks e seis conjuntos, a mais rápida é sempre a mais econômica; com núcleos fixos, a razão de energia acompanha a de tempo
Peso da memória	Em parte	DRAM secundária na configuração padrão (0,6% a 4,5%), mas 31% a 45% na frequência mínima: depende do estado do processador
Bibliotecas de terceiros	Confirmada	regex-redux tem a menor razão em todos os conjuntos; pidigits está quebrado também nos dados de van Kempen et al.

Conclusão

Pereira et al.: a ordem, não o fator

C++ é mais rápido e mais econômico em oito de nove benchmarks, e a ordem se reproduz. O fator cai de 56,6 para 32,8 com o mesmo código. A análise falha onde van Kempen aponta: medidor, paralelismo desigual e a variação da potência atribuída à linguagem.

van Kempen et al.: o modelo, não os coeficientes

A frequência fixa, potência igual para todos os pares a menos de 5%. Núcleos e memória governam a potência com as formas do artigo, mas o limiar para paralelizar é 44% no desktop, não 13%, e o peso da DRAM depende do estado do processador.

Com interpretador e paralelismo fixos, Python fica em torno de 25 vezes C++ em tempo e em energia. A linguagem afeta a energia por meio do tempo; o fator de Pereira et al. mede também o interpretador, os núcleos e a máquina.

Obrigado

Código, dados e scripts:

github.com/maycon-freitas/reproduction-energy-languages

Maycon Freitas · PPgSC/UFRN